Lab Eight – Probability

Tips

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with "**Solution**:".
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.
- Make sure to use "enter" to make your code readable, so it doesn't run off the page. I switched the Quarto document to automatically render to .pdf, so you can see what you're submitting by default. You can switch back for the highlighting by moving the html block above the pdf block in the YAML header.
- Don't forget that you can copy-and-paste code when you're doing something similar as we did in class!
- Make sure to plot all computed probabilities. This is good `ggplot` practice AND will help make sure you don't make a mistake when computing the probability.

1 The Binomial and Negative Binomial Distributions

1.1 The Binomial Distribution

In many introductory courses, we make the simplifying assumption of independence in Bernoulli trials. For example, when playing best two-out-of-three legs in a darts match, we would assume Player A has an 70% chance of winning each leg. We write:

$$X \sim \text{Binomial}(n = 3, p = 0.70)$$

where

- X is the number of legs won
- n is the number of trials
- p is the probability of the event of interest

Of note, it is not necessarily the case that the players play all three legs – if either player wins legs one and two the match ends.

There are eight possible outcomes for Player A:

- (win, win, "win") $\longrightarrow$ Player A wins
- (win, win, "lose") $\longrightarrow$ Player A wins
- (win, lose, win) $\longrightarrow$ Player A wins
- (win, lose, lose) $\longrightarrow$ Player A loses
- (lose, win, win) $\longrightarrow$ Player A wins
- (lose, win, lose) $\longrightarrow$ Player A loses
- (lose, lose, "win") $\longrightarrow$ Player A loses
- (lose, lose, "lose") $\longrightarrow$ Player A loses

Note: I use quotations above to denote outcomes where the last match isn't actually played.

Use the Binomial distribution to compute the probability that Player A wins the match. Make sure write out your probability statement and work to write it in terms of the distribution function (PMF/CDF) to plot the probability using `ggplot`.

Solution

Here, I am looking for $P(X \geq 2)$, the probability that player A wins at least two matches. I can do

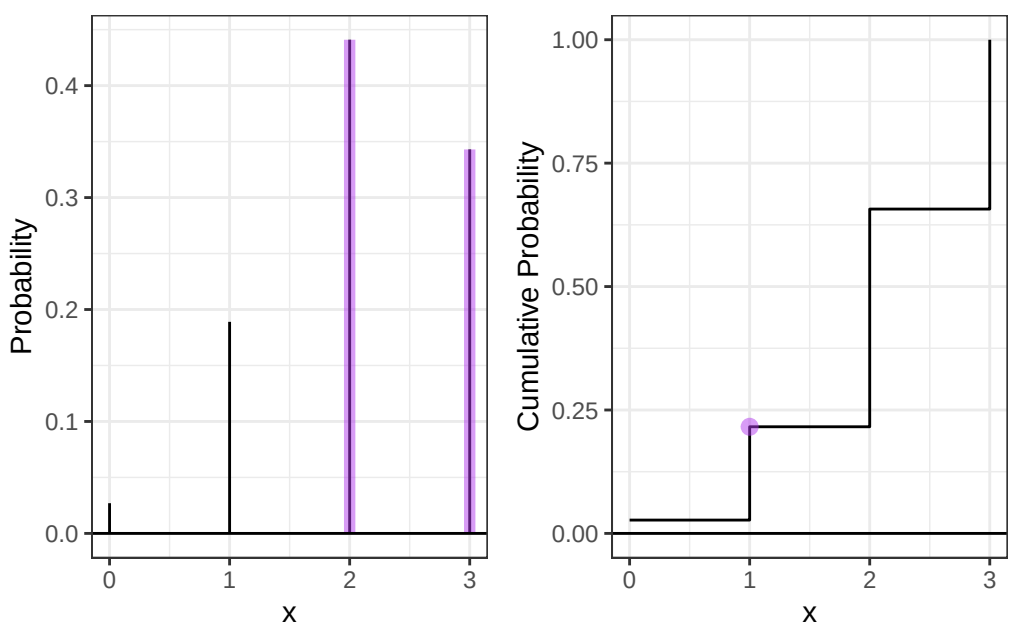
$$\begin{aligned} P(X \geq 2) &= 1 - P(X < 2) \\ &= 1 - P(X \leq 1) \\ &= 1 - F_X(1) \end{aligned}$$

Thus, I can use the CDF function (pbinom) to easily calculate this probability. I've also plotted both CDF and PMF below, highlighting the probability we were calculating for.

```
win_percent = 1 - pbinom(q=1, prob = 0.7, size = 3)
(win_percent)
```

```
[1] 0.784
```

```
ggdat = tibble(x = 0:3) |>
  mutate(PMF = dbinom(x=x, prob=0.7, size = 3),
         CDF = pbinom(q=x, prob = 0.7, size = 3))

PMF.plot = ggplot() +
  geom_linerange(data = ggdat,
                 aes(x = x, ymin = 0, ymax = PMF)) +
  geom_linerange(data = ggdat |> filter(x>=2),
                 aes(x = x, ymin = 0, ymax = PMF),
                 color = "darkorchid2", linewidth=2, alpha=0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks=0:4) +
  ylab("Probability") +
  theme_bw()

CDF.plot = ggplot() +
  geom_step(data=ggdat, aes(x = x, y = CDF)) +
  geom_point(data=ggdat |> filter(x==1),
             aes(x = x, y = CDF),
             color="darkorchid2", size = 2.5, alpha =0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks=0:4) +
  ylab("Cumulative Probability") +
  theme_bw()

PMF.plot | CDF.plot
```



1.2 The Negative Binomial Distribution

In intermediate courses, we learn the Negative Binomial distribution that we use to model the number of "failures" until the rth success. Note that this approach still makes the simplifying assumption of independence in Bernoulli

trials.

Specify the two cases necessary to compute the probability. Fully explain these probability statements in the context of a best two-out-of-three match and the Negative Binomial distribution. Finally, compute the probability that Player A wins the match and show you get the same result as the Binomial approach. Make sure write out your probability statement and work to write it in terms of the distribution function (PMF/CDF) to plot the probability using `ggplot`.

Solution

There are two cases for the two out of three match. The first case is player A wins the first two matches in a row, and technically wins immediately. The second case is player A has exactly one loss before their two wins in order to win the whole game. Therefore, let X be the number of failures before Player A gets two wins. So, the probability that player A wins, where X follows a negative binomial distribution, is $P(X \leq 1)$.

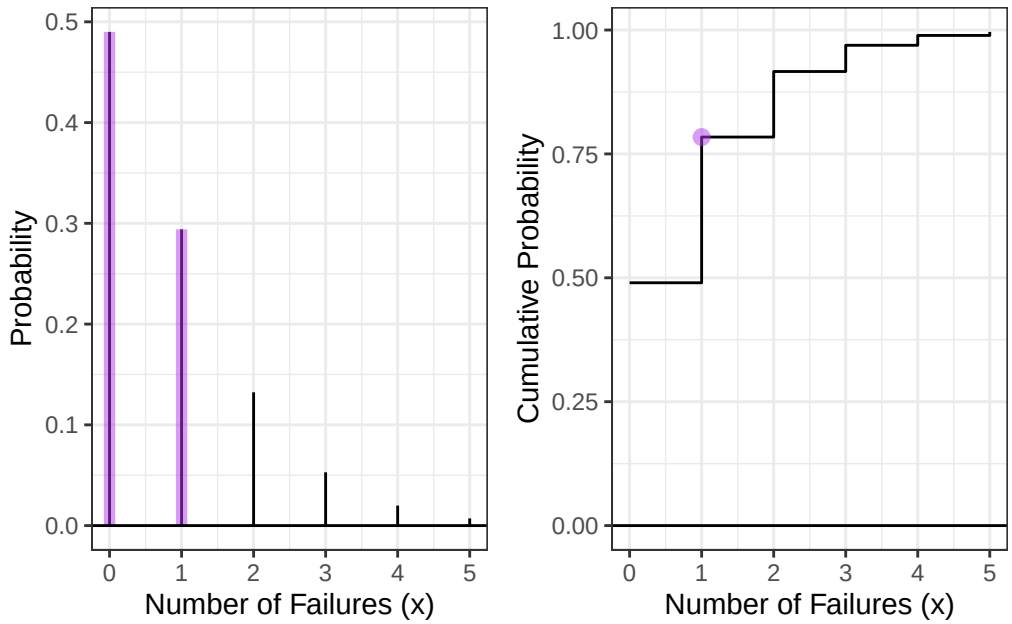
$$P(X \leq 1) = F_X(1)$$

I get the same result as I did for the binomial distribution.

```r
win_percent = pnbinom(q=1, prob=0.7, size = 2)
(win_percent)
```

```
[1] 0.784
```

```r
ggdat = tibble(x = 0:5) |>
  mutate(PMF = dnbinom(x=x, prob=0.7, size = 2),
         CDF = pnbinom(q=x, prob = 0.7, size = 2))

PMF.plot = ggplot() +
  geom_linerange(data = ggdat,
                 aes(x = x, ymin = 0, ymax = PMF)) +
  geom_linerange(data = ggdat |> filter(x<=1),
                 aes(x = x, ymin = 0, ymax = PMF),
                 color = "darkorchid2", linewidth=2, alpha=0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("Number of Failures (x)", breaks=0:5) +
  ylab("Probability") +
  theme_bw()

CDF.plot = ggplot() +
  geom_step(data=ggdat, aes(x = x, y = CDF)) +
  geom_point(data=ggdat |> filter(x==1),
             aes(x = x, y = CDF),
             color="darkorchid2", size = 2.5, alpha =0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("Number of Failures (x)", breaks=0:5) +
  ylab("Cumulative Probability") +
  theme_bw()

PMF.plot | CDF.plot
```



2 The Beta Binomial Distribution

One approach to generalize this modeling is to use a Beta-Binomial distribution. This distribution arrises when we let p be a random variable that follows a Beta distribution:

$$P \sim \text{Beta}(\alpha, \beta)$$
$$X \sim \text{Binomial}(n, P).$$

This distribution is not part of base `R`, but is available using the `_bbinom` functions from the `extraDistr` package (Wolodzko 2026).

The primary benefit is that the Beta-Binomial enables us to specify some uncertainty about the skill of Player A, as p does not have to be fixed (e.g., $p = 0.70$). Suppose the result of the best two-out-of-three match follows a Beta-Binomial distribution where $\alpha = 7$ and $\beta = 3$.

2.1 The Beta Part

1. **Plot the corresponding Beta distribution. Interpret the plot in the context of a best two-out-of-three match and the Binomial distribution.**
   Hint: The Beta distribution is available in base `R` via `_beta()`.

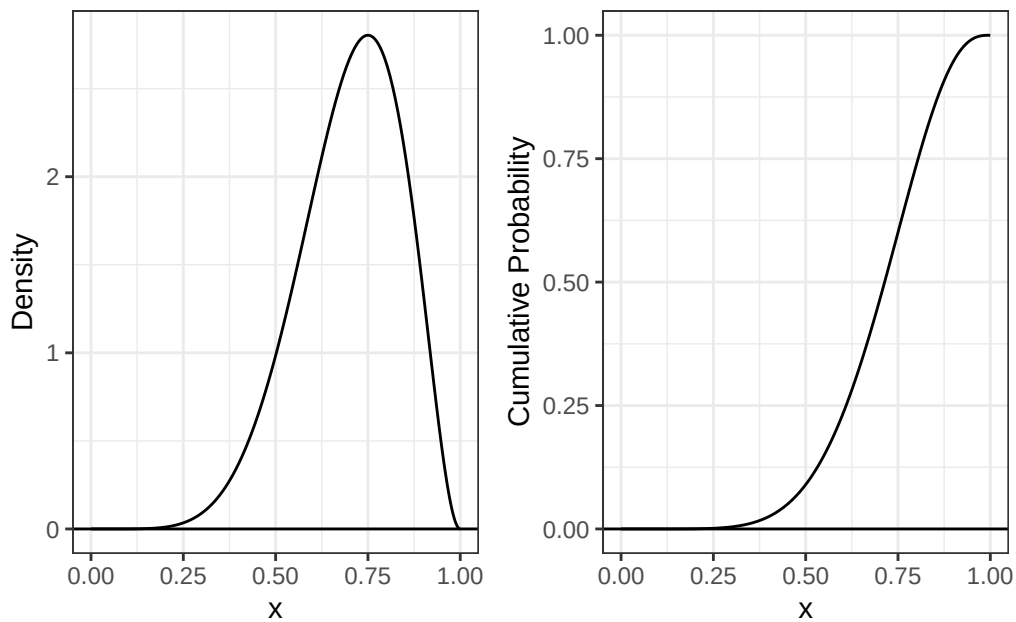
Solution

The distribution models the uncertainty of Player A's leg-win probability. Instead of it being a fixed value at 0.7, the distribution is centered around 0.7 with some variance such that the leg-win probability could be slightly above or below that.

```
ggdat = tibble(x = seq(0, 1, length.out=1000)) |>
  mutate(PDF = dbeta(x = x, 7, 3),
         CDF = pbeta(q = x, 7, 3))

PDF.plot = ggplot(ggdat) +
  geom_line(aes(x = x, y = PDF)) +
  geom_hline(yintercept = 0) +
  xlab("x") +
  ylab("Density") +
  theme_bw()

CDF.plot = ggplot(ggdat) +
  geom_line(aes(x = x, y = CDF)) +
  geom_hline(yintercept = 0) +
  xlab("x") +
```

```
  ylab("Cumulative Probability") +
  theme_bw()

PDF.plot | CDF.plot
```

Figure 1: Beta Distribution



1. How does the Beta incorporate uncertainty about the skill of Player A? **Compute the mean and variance of the distribution.**
   Hint: You will find the `integrate(...)` function helpful here.

$$E(P) = \mu_P = \int_0^1 p f_P(p) dp$$
$$var(P) = \sigma_P^2 = \int_0^1 (p - \mu_P)^2 f_P(p) dp$$

Solution

The Beta incorporates uncertianty through the variance of the distribution which slightly varies the skill of Player A. I make separate functions for each mean and variance, and then I integrate each from 0 to 1. I had an issue here because I didn't realize the integrate function returns characters so I had to specify my value using value.

```
#mean
f_mean = function(p){
  p * dbeta(p, 7, 3)
}
#beta mean
beta_mean = integrate(f_mean, lower = 0, upper = 1)
(beta_mean)
```

```
0.7 with absolute error < 7.8e-15
```

```
#var
f_var = function(p){
  dbeta(p,7,3) * (p-beta_mean$value)^2
}
#beta var
beta_var = integrate(f_var, lower = 0, upper = 1)
(beta_var)
```

```
0.01909091 with absolute error < 2.1e-16
```

1. **What is the probability that Player A has a win probability of exactly 0.70? Make sure to write out your probability statement and work to write it in terms of the distribution function (PDF/CDF) to plot the probability using `ggplot`.**
   Hint: Think carefully about the properties of continuous distributions here.

Solution

Because X is a continuous random variable on a Beta distribution, the probability of any exact single value is 0. Density is not the same thing as probability here.

1. **What is the probability that Player A has a win probability of at least 0.50? Make sure write out your probability statement and work to write it in terms of the distribution function (PDF/CDF) to plot the probability using `ggplot`.**

Solution

$$P(X \geq 0.5) = 1 - P(X \leq 0.5) = 1 - F_X(0.5)$$

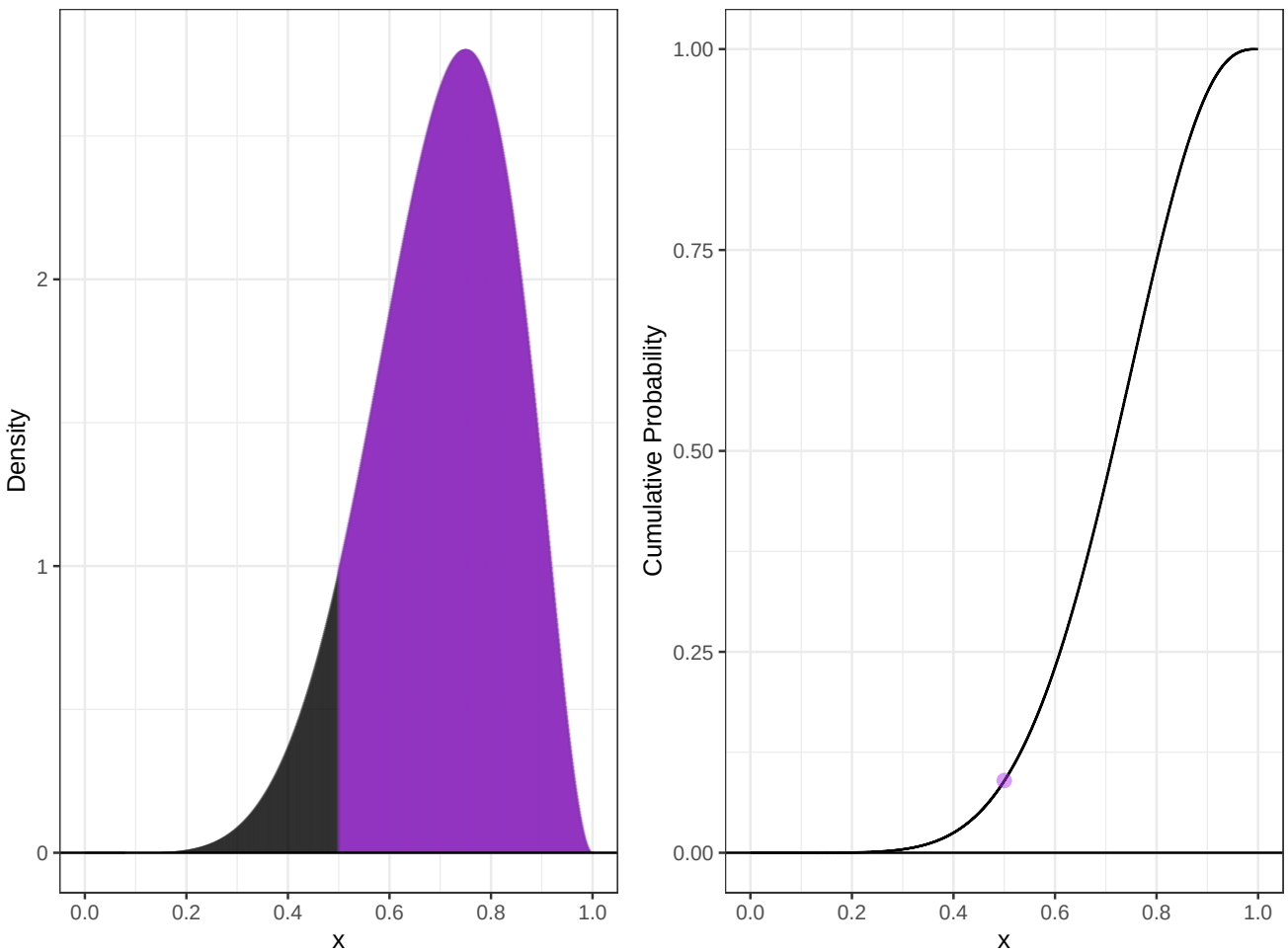
I use a standard probability statement to find I just need to use the CDF function to find this probability. I shade the region on the density graph and mark the point on the CDF plot.

```r
#atleast 50 win prob
prob_0.5 = 1 - pbeta(q = 0.5, shape1 = 7, shape2 = 3)
prob_0.5
```

```
[1] 0.9101562
```

```r
ggdat = tibble(x = seq(0, 1, length.out = 1000)) |>
  mutate(
    PDF = dbeta(x = x, 7, 3),
    CDF = pbeta(q = x, 7, 3)
  )

PDF.plot = ggplot() +
  geom_linerange(data=ggdat,aes(x = x, ymin = 0, ymax = PDF),
                 alpha = 0.3) +
  geom_linerange(data=ggdat |> filter(x >=0.5),
                aes(x = x, ymin = 0, ymax = PDF),
                color = "darkorchid2", linewidth=1, alpha=0.15) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks=seq(0,1,by=0.2)) +
  ylab("Density") +
  theme_bw()

CDF.plot = ggplot() +
  geom_step(data = ggdat, aes(x = x, y = CDF)) +
  geom_point(data = ggdat |> filter(x==1),
            aes(x = 0.5, y = pbeta(q=0.5,7,3)),
            color="darkorchid2", size = 2.5, alpha =0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks=seq(0,1,by=0.2)) +
  ylab("Cumulative Probability") +
  theme_bw()

PDF.plot | CDF.plot
```

Figure 2: At least 50% win probability



1. **What is the 90th percentile of Player A's win probability based on this Beta distribution? Make sure write out your probability statement and work to write it in terms of the distribution function (PDF/CDF) to plot the percentile using `ggplot`.**

Solution

For the 90th percentile I set up as:

$$P(X \leq q) = 0.90 = F_X(q)$$

Solving for q:

$$q = F_X^{-1}(0.90)$$

We can easily use the inverse CDF function for this.

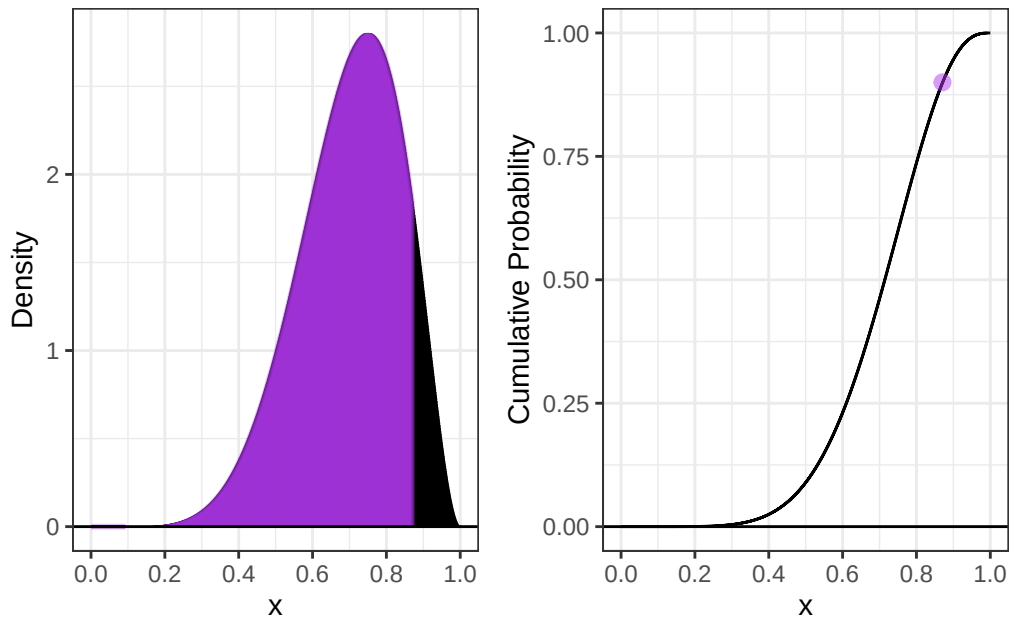
```
#90th percentile win prob
q_90 = qbeta(p = 0.90, shape1 = 7, shape2 = 3)
q_90
```

```
[1] 0.8705027
```

```
ggdat = tibble(x = seq(0, 1, length.out = 1000)) |>
  mutate(
    PDF = dbeta(x = x, 7, 3),
    CDF = pbeta(q = x, 7, 3)
  )

PDF.plot = ggplot() +
```

```r
  geom_linerange(data = ggdat,
                 aes(x = x, ymin = 0, ymax = PDF)) +
  geom_linerange(data = ggdat |> filter(x <= q_90),
                 aes(x = x, ymin = 0, ymax = PDF),
                 color = "darkorchid2", linewidth = 1, alpha = 0.15) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = seq(0, 1, by = 0.2)) +
  ylab("Density") +
  theme_bw()

CDF.plot = ggplot() +
  geom_step(data = ggdat, aes(x = x, y = CDF)) +
  geom_point(aes(x = q_90, y = 0.90),
             color = "darkorchid2", size = 2.5, alpha = 0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = seq(0, 1, by = 0.2)) +
  ylab("Cumulative Probability") +
  theme_bw()

PDF.plot | CDF.plot
```

Figure 3: 90th percent



2.2 The Beta-Binomial Part

Compute the probability that Player A wins the match. Compare your result to the Binomial and Negative Binomial results. Make sure write out your probability statement and work to write it in terms of the distribution function (PMF/CDF) to plot the probability using `ggplot`

Solution

For X being the number of legs won by Player A in the game, under the beta-binomial model, X follows the beta-bionomial distribution. Player A wins if they win at least 2 out of the 3 legs. Thus,

$$P(X \geq 2) = 1 - P(X \leq 1) = 1 - F_X(1)$$

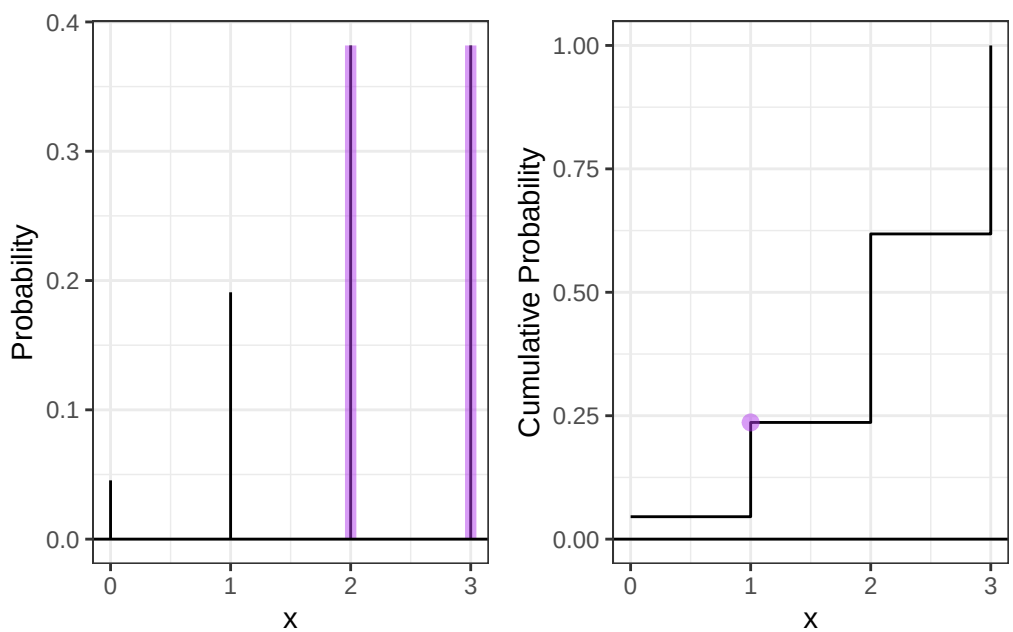
This is similar to the binomial model, but now our leg winning probability varies slightly accoridng to the beta distribution.

```r
#our varying win prob
win = 1 - pbbinom(q = 1, size = 3, alpha = 7, beta = 3)
win
```

```
[1] 0.7636364
```

```r
ggdat = tibble(x = 0:3) |>
  mutate(
    PMF = dbbinom(x = x, size = 3, alpha = 7, beta = 3),
    CDF = pbbinom(q = x, size = 3, alpha = 7, beta = 3)
  )

PMF.plot = ggplot() +
  geom_linerange(data = ggdat,
                 aes(x = x, ymin = 0, ymax = PMF)) +
  geom_linerange(data = ggdat |> filter(x >= 2),
                 aes(x = x, ymin = 0, ymax = PMF),
                 color = "darkorchid2", linewidth = 2, alpha = 0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:3) +
  ylab("Probability") +
  theme_bw()

CDF.plot = ggplot() +
  geom_step(data = ggdat, aes(x = x, y = CDF)) +
  geom_point(data = ggdat |> filter(x == 1),
             aes(x = x, y = CDF),
             color = "darkorchid2", size = 2.5, alpha = 0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:3) +
  ylab("Cumulative Probability") +
  theme_bw()

PMF.plot | CDF.plot
```

Figure 4: Beta-Binomial



My winning probability for the beta-binomial model is ~ 76% compared to the binomial and negative binomial results of ~78%. This discrepancy is most likely due to Player A's varying win probability.

2.3 The Why

Check the expected value and variance of each distribution.

Analytically, we have

$$E(X) = np \qquad var(X) = np(1-p) \qquad \text{(Binomial Distribution)}$$

$$E(X) = n\left(\frac{\alpha}{\alpha+\beta}\right) \qquad var(X) = \frac{n\alpha\beta(\alpha+\beta+n)}{(\alpha+\beta)^2(\alpha+\beta+1)} \qquad \text{(Beta-Binomial Distribution)}$$

For each distribution, compute the mean number of legs won for player A.
Hint: You can perform the sum over the finite support set here.

$$E(X) = \mu_X = \sum_{i=0}^{3} i f_X(i)$$

$$var(X) = \sigma_X^2 = \sum_{i=0}^{3} (i-\mu_X)^2 f_X(i)$$

Solution

```r
n = 3
p = 0.7

#expected binomial value
E_x_binom = n*p
E_x_binom
```

```
[1] 2.1
```

```r
#binomial var
var_x_binom = n* p * (1-p)
var_x_binom
```

```
[1] 0.63
```

```r
alpha = 7
beta = 3

#expected beta binom value
E_x_betabinom = n * (alpha/(alpha+beta))
E_x_betabinom
```

```
[1] 2.1
```

```r
#beta binom var
var_x_betabinom = (n*alpha*beta*(alpha+beta+n))/((alpha+beta)^2*(alpha+beta+1))
var_x_betabinom
```

```
[1] 0.7445455
```

```r
x = 0:3

binom_PMF = dbinom(x = x, size = 3, prob = 0.7)
betabinom_PMF = dbbinom(x = x, size = 3, alpha = 7, beta = 3)

#expected
binom_mean = sum(x * binom_PMF)
binom_mean
```

```
[1] 2.1
```

```r
betabinom_mean = sum(x * betabinom_PMF)
betabinom_mean
```

```
[1] 2.1
```

```r
#standard deviation
binom_std = sqrt(sum((x - binom_mean)^2 * binom_PMF))
binom_std
```

```
[1] 0.7937254
```

```r
betabinom_std = sqrt(sum((x - betabinom_mean)^2 * betabinom_PMF))
betabinom_std
```

```
[1] 0.8628705
```

The mean number of legs for both the binomial and beta-binomial distribution is 2.1. However, the binomial variance is 0.63, while the beta-binomial variance is higher at 0.74. This is higher variability is because the beta-binomial systems allows the leg-win probability to vary slightly.

3 Altham's Multiplicative Binomial Distribution

While the Beta-Binomial enables us to model cases where winning has a clustering effect (e.g., a "hot hand").

3.1 Probability Mass Function

The mass function for this distribution is below. While it used to be available in the `MBD` package for `R`, this package is no longer available.

$$f_X(x) = \frac{1}{c}\binom{n}{x} p^x (1-p)^{n-x} \theta^{x(n-x)} I(x \in \{0, 1, ..., n\})$$

where

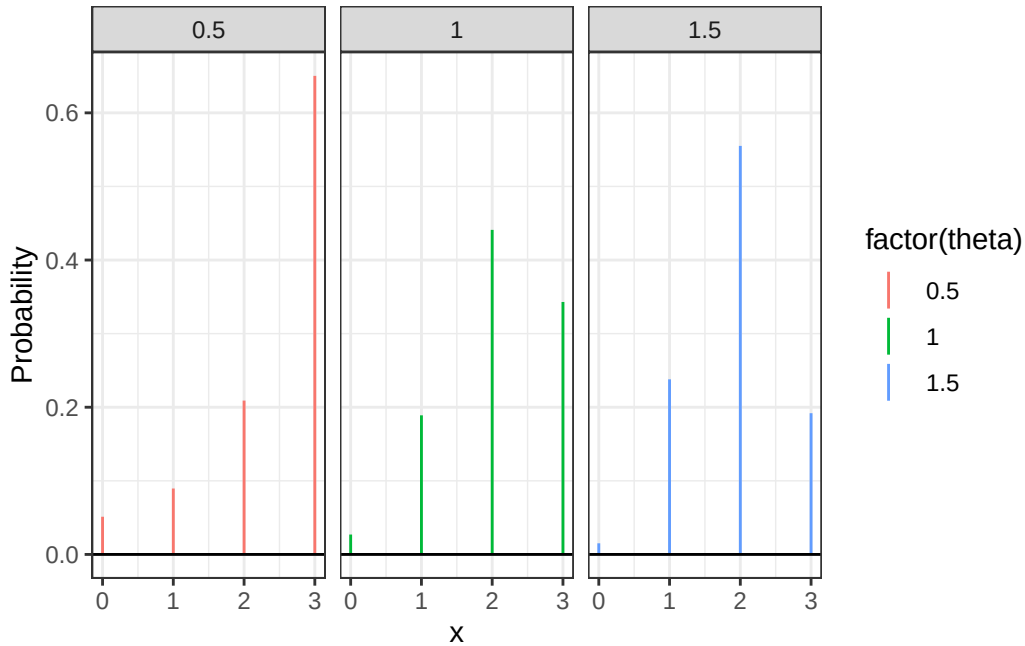
$$c = \sum_{i=0}^{n} \binom{n}{i} p^i (1-p)^{n-i} \theta^{i(n-i)}$$

Write a function in `R` called `daltbinom()` for computing the PMF of this distribution. Use your function to plot the distribution where $p = 0.70$ where $\theta = 0.50$, $\theta = 1$, and $\theta = 1.50$. Comment on the differences in the context of a best two-out-of-three match

Solution

```r
#made my function here
daltbinom = function(x, p, theta, n){
  i = 0:n

  c = sum((choose(n,i))*(p^i)*(1-p)^(n-i)*theta^(i*(n-i)))

  pmf = 1/c * choose(n,x) * p^x * (1-p)^(n-x)*theta^(x*(n-x))

  #use logical for indicator
  return(pmf * (x %in% i))
}

#created my data table
ggdat = tibble(
  x = rep(0:3, times = 3),
  theta = rep(c(0.5,1,1.5),each = 4)
) |>
  mutate(
    PMF = case_when(
      theta == 0.5 ~ daltbinom(x=x, p = 0.7, theta = 0.5, n = 3),
      theta == 1 ~ daltbinom(x = x, p = 0.7, theta = 1, n = 3),
      theta == 1.5 ~ daltbinom(x=x, p = 0.7, theta = 1.5, n = 3)
    )
  )

#plot
PMF.plot = ggplot() +
  geom_linerange(data = ggdat,
                 aes(x = x, ymin = 0, ymax = PMF, color = factor(theta))) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:3) +
  ylab("Probability") +
  theme_bw()+
  facet_wrap(~factor(theta))

PMF.plot
```

Figure 5: PMF for theta = 0.5, 1, 1.5 and p = 0.7



Based on the PMF plot, we see that as we vary theta lower, we get a more 'hot-handed' effect, where you're more likely to win more hands the better you're doing, so we get a clumpy effect. At theta = 1, we get a normal binomial distribution, as we find there to be no effect on the PMF at that number. At higher thetas, we see a less clumpy effect where the number of legs won is more centered around 2, and thus the hot-hand effect has the opposite effect, where wins result in less wins, and it's more weighted towards Player A's leg-win probability, p. Thus, we get more 2-1 games.

3.2 Cumulative Distribution Function

Use your `daltbinom()` function to create a `paltbinom()` function for computing the CDF of this distribution. Use your function to plot the distribution where $p = 0.70$ where $\theta = 0.50$, $\theta = 1$, and $\theta = 1.50$.

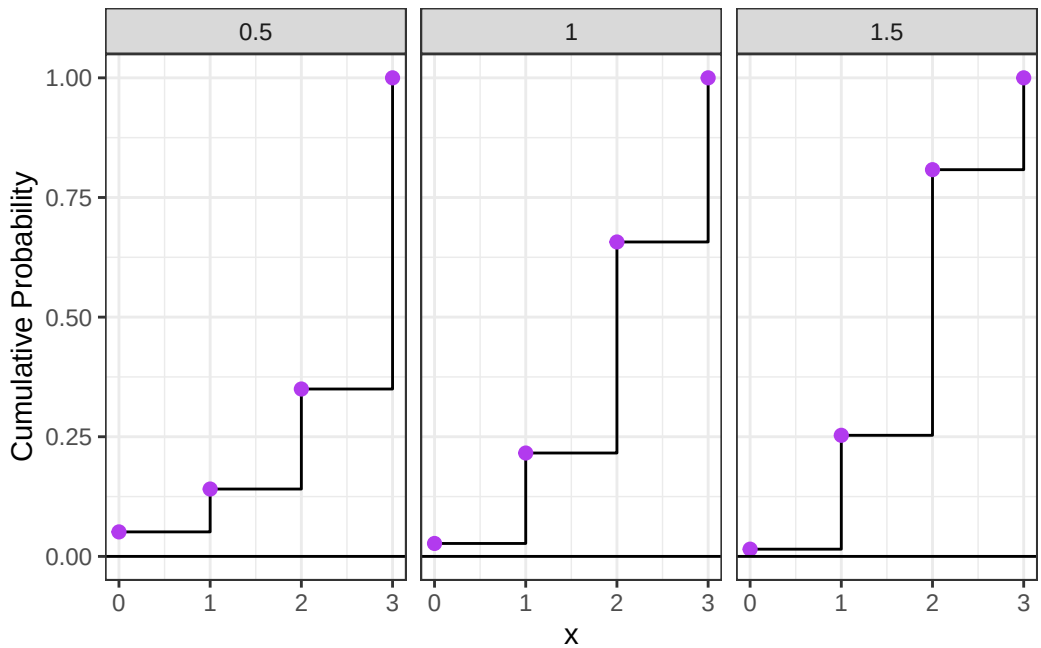
Solution

Since the CDF is just the probability that X is less than or equal to some value, I can compute it by summing the PMF from 0 up to x.

```
paltbinom = function(x, p, theta, n){
  sapply(x, function(z){ #applies function for each 0 to z in x
    sum(daltbinom(x = 0:z, p = p, theta = theta, n = n))}) #sums PMF values from 0 to z
}

#created my data table
ggdat = tibble(
  x = rep(0:3, times = 3),
  theta = rep(c(0.5,1,1.5),each = 4)
) |>
  mutate(
    CDF = case_when(
      theta == 0.5 ~ paltbinom(x=x, p = 0.7, theta = 0.5, n = 3),
      theta == 1 ~ paltbinom(x = x, p = 0.7, theta = 1, n = 3),
      theta == 1.5 ~ paltbinom(x=x, p = 0.7, theta = 1.5, n = 3)
    )
  )

#plot
CDF.plot = ggplot() +
  geom_step(data = ggdat, aes(x = x, y = CDF)) +
  geom_point(data = ggdat,
             aes(x = x, y = CDF),
             color = "darkorchid2", size = 2) +
```

```
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:3) +
  ylab("Cumulative Probability") +
  theme_bw() +
  facet_wrap(~factor(theta))

CDF.plot
```

Figure 6: CDF for theta = 0.5, 1, 1.5 and p = 0.7



Based on the CDF plot we see that changing our theta values affects where the cumulative probability clumps up. At lower theta values, probability clumps reflecting the hot-hand effect. Once again, at theta = 1, we get a normal binomial distribution. At greater theta values, the clumpy effect is reduced, with more probability at middle values, thus more likely heading to 2-1 outcomes.

3.3 The Winning Probability

Compute the probability that Player A wins the match using Altham's Multiplicative Binomial Distribution. Compare your result to the Binomial and Negative Binomial results. Compare your results to the Beta-Binomial model, too. Make sure write out your probability statement and work to write it in terms of the distribution function (PMF/CDF) to plot the probability using `ggplot`.

Solution

Once again, lets let X be the number of legs Player A wins in a 2-out-of-3 match, where X follows Altham's Multiplicative Binomial Distribution. Since Player A wins the game if they win at least 2 legs, we are looking for $P(X \geq 2)$. This is the same as:

$$P(X \geq 2) = 1 - P(X \leq 1) = 1 - F_X(1)$$

.

```
#each theta case here
win_prob_0.5 = 1 - paltbinom(x = 1, p = 0.7, theta = 0.5, n = 3)
win_prob_0.5
```

```
[1] 0.8592417
```

```
win_prob_1 = 1 - paltbinom(x = 1, p = 0.7, theta = 1, n = 3)
win_prob_1
```

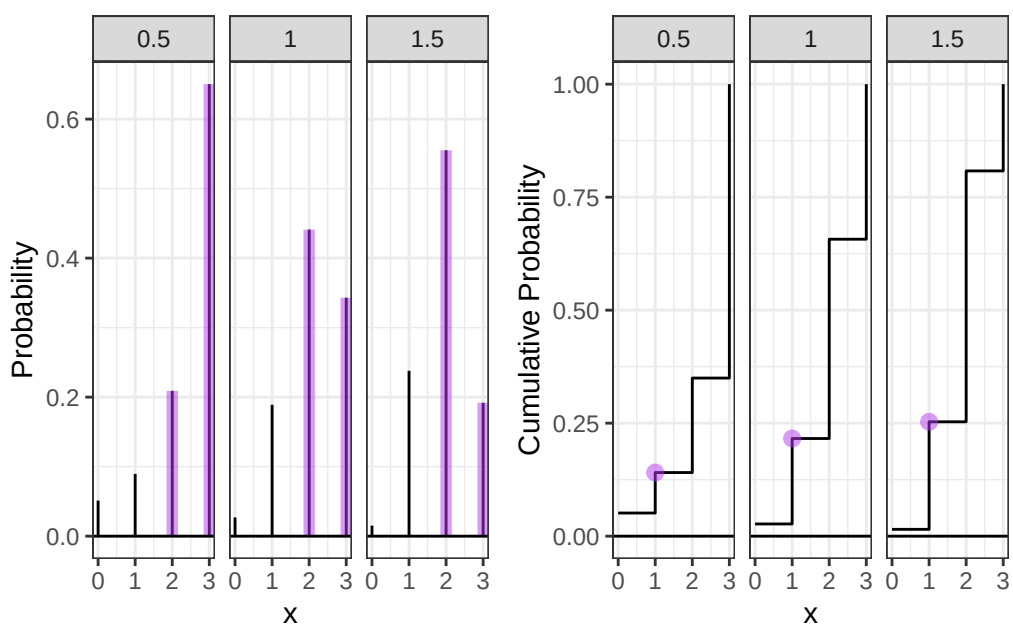
```
[1] 0.784
```

```r
win_prob_1.5 = 1 - paltbinom(x = 1, p = 0.7, theta = 1.5, n = 3)
win_prob_1.5
```

```
[1] 0.746993
```

```r
ggdat = tibble(
  x = rep(0:3, times = 3),
  theta = rep(c(0.5, 1, 1.5), each = 4)
) |>
  mutate(
    PMF = case_when(
      theta == 0.5 ~ daltbinom(x = x, p = 0.7, theta = 0.5, n = 3),
      theta == 1 ~ daltbinom(x = x, p = 0.7, theta = 1, n = 3),
      theta == 1.5 ~ daltbinom(x = x, p = 0.7, theta = 1.5, n = 3)
    ),
    CDF = case_when(
      theta == 0.5 ~ paltbinom(x = x, p = 0.7, theta = 0.5, n = 3),
      theta == 1 ~ paltbinom(x = x, p = 0.7, theta = 1, n = 3),
      theta == 1.5 ~ paltbinom(x = x, p = 0.7, theta = 1.5, n = 3)
    )
  )

PMF.plot = ggplot() +
  geom_linerange(data = ggdat,
                 aes(x = x, ymin = 0, ymax = PMF)) +
  geom_linerange(data = ggdat |> filter(x >= 2),
                 aes(x = x, ymin = 0, ymax = PMF),
                 color = "darkorchid2", linewidth = 2, alpha = 0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:3) +
  ylab("Probability") +
  theme_bw() +
  facet_wrap(~factor(theta))

CDF.plot = ggplot() +
  geom_step(data = ggdat, aes(x = x, y = CDF)) +
  geom_point(data = ggdat |> filter(x == 1),
             aes(x = x, y = CDF),
             color = "darkorchid2", size = 2.5, alpha = 0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:3) +
  ylab("Cumulative Probability") +
  theme_bw() +
  facet_wrap(~factor(theta))

PMF.plot | CDF.plot
```

Figure 7: Altham's Multiplicative Binomial Distribution with theta = 0.5, 1,r 1.5 and p = 0.7



When theta is 1, the model gives the same result as in the ordinary binomial distribution. When theta is 0.5, the distribution is more clumpy so it is more likely Player A will win as wins influence more wins, already being influenced by Player A's 70% leg-win chance. When theta is 1.5, distribution is more centerd around the center, thus a greater 2-1 outcome, thus influencing a relatively lower win chance for Player A.

Both the binomial and negative binomial gave a win probability of 0.784, while the beta-binomial model gave a lower win probability of 0.7636. For Altham's it depends on the theta value used. For lower theta values the win probability is higher than the rest of the models. For higher theta values the win probability is lowered even below all the other models' win probability. Altham's model better models the case where legs have a dependent, not independent, effect on other legs.

4 Altham's Multiplicative Beta-Binomial Distribution

You may have wondered – "Can we do the Beta thing with Altham's version of the Binomial Distribution?" The answer is yes!

4.1 Probability Mass Function

The mass function for this distribution is below.

$$f_X(x) = \left[\int_0^1 dp \left(\frac{1}{c}\binom{n}{x} p^x (1-p)^{n-x} \theta^{x(n-x)}\right) \left(\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} p^{\alpha-1}(1-p)^{\beta-1}\right)\right] I(x \in \{0, 1, ..., n\})$$

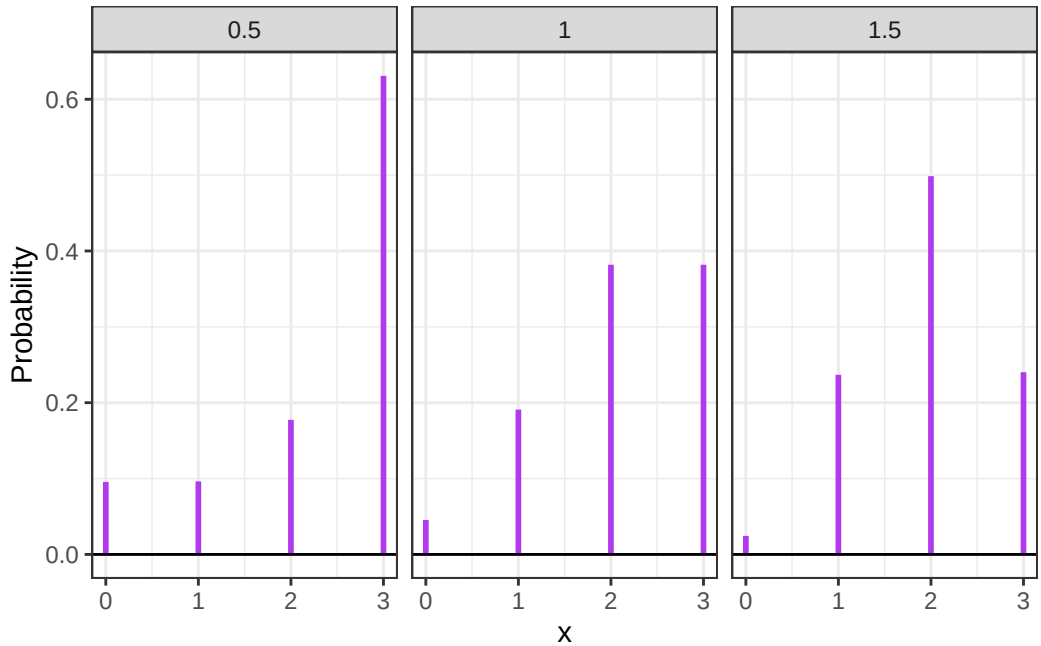
Write a function in `R` called `daltbbinom()` for computing the PMF of this distribution. Use your function to plot the distribution where $\alpha = 7$ and $\beta = 3$ where $\theta = 0.50$, $\theta = 1$, and $\theta = 1.50$. Comment on the differences in the context of a best two-out-of-three match.

Hint: You can use the `integrate()` in `R` to compute the integral. Note that this requires the function to be vectorized – it should be unless you have added any block conditionals.

Solution

```r
daltbbinom = function(x, alpha, beta, theta, n){
  #I had an issue because integrate() passes in multiple values of p at once,
  #so I used a second sapply() to evaluate one p-value at a time
  # hence the double sapply, i wont lie i had to consult oscar for this bc i didnt understand why my PMF was not adding up to 1 before.

  sapply(x, function(z){ #apply from 0 to z in x
    f_x = function(p){
      sapply(p, function(m){ #apply from 0 to m in p (ik my naming convention sucks, i had to try a bunch of things for this to work out)
        daltbinom(x = z, p = m, theta = theta, n = n) * dbeta(m, alpha, beta)
      })
    }
    integrate(f_x, lower = 0, upper = 1)$value
  })
}

ggdat = tibble(
  x = rep(0:3, times = 3),
  theta = rep(c(0.5, 1, 1.5), each = 4)
) |>
  mutate(
    PMF = case_when(
      theta == 0.5 ~ daltbbinom(x = x, alpha = 7, beta = 3, theta = 0.5, n = 3),
      theta == 1 ~ daltbbinom(x = x, alpha = 7, beta = 3, theta = 1, n = 3),
      theta == 1.5 ~ daltbbinom(x = x, alpha = 7, beta = 3, theta = 1.5, n = 3)
    )
  )

PMF.plot = ggplot() +
  geom_linerange(data = ggdat,
                 aes(x = x, ymin = 0, ymax = PMF),
                 linewidth = 1, color = "darkorchid2") +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:3) +
  ylab("Probability") +
  theme_bw() +
  facet_wrap(~factor(theta))

PMF.plot
```

Figure 8: PMF for Altham's Multiplicative BB Distribution with theta = 0.5, 1, 1.5, alpha = 7, and beta = 3



When theta is 0.5, we see the same clumpy effect towards 3 where Player A is much more likely to win all 3 games. When theta is 1, we reduce back down to the Beta-binomial model. When theta is 1.5, the distribution gets to be less clumpy and more centered around 2 wins, results in 2-1 matches more.

5 A Comparison

Compute the probabilities for all outcomes for Player A.

- Loses 0-2 (LL"L", LL"W")
- Loses 1-2 (WLL, LWL)
- Wins 2-0 (WW"W", WW"L")
- Wins 2-1 (LWW, WLW)

Note: You'll need to define these outcomes in terms of the random variable.

Create a bar plot (or column plot) using `ggplot` to show the probability of each outcome under the following conditions:

- **Binomial**
- **Negative Binomial**
  Note: You can compute the lose probabilities the same way as the win probabilities using success probability $(1 - p)$
- **Beta-Binomial**
- **Althman Binomial with $\theta = 0.50$ and $\theta = 1.5$**
- **Althman Beta-Binomial with $\theta = 0.50$ and $\theta = 1.5$**

Comment on the differences in the context of a best two-out-of-three match.

Solution

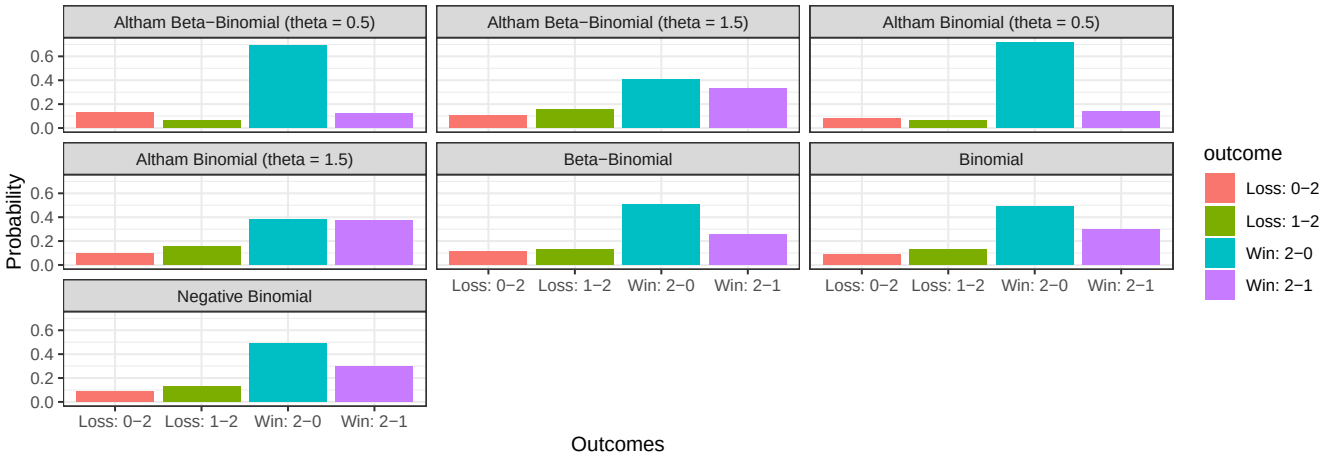
I let X be the total number of legs won by Player A out of the three possible leg outcomes, including when the third leg is unplayed. I started with the probability distribution for total wins out of the 3 possible legs, and then split those into the actual match outcomes. Since some three-leg outcomes include cases where the third leg would not actually be played (WWW or WWL), I redistributed those probabilities to match the real results (like we made the

table after class) That way I could compare all the models using the exact outcome categories, instead of reordering and making things more convulated.

```r
outcomes = c("Loss: 0-2", "Loss: 1-2", "Win: 2-0", "Win: 2-1")
alpha = 7
beta = 3
p=0.7

#binomial
probs = dbinom(x = 0:3, size = 3, prob = 0.7)

binom_dat = tibble(
  outcome = outcomes,
  prob = c((probs[1]+1/3*probs[2]),2/3*probs[2], probs[4]+1/3*probs[3], 2/3*probs[3]),
  model = "Binomial"
)

#negative binomial
#i use 1-p for losses (overall)
#i use p for wins (overall)
nbinom_dat = tibble(
  outcome = outcomes,
  prob = c(
    dnbinom(x = 0, size = 2, prob = 1-p),
    dnbinom(x = 1, size = 2, prob = 1-p),
    dnbinom(x = 0, size = 2, prob = p),
    dnbinom(x = 1, size = 2, prob = p)),
  model = "Negative Binomial"
)

#beta binom
probs = dbbinom(x = 0:3, size = 3, alpha = alpha, beta = beta)

bbinom_dat = tibble(
  outcome = outcomes,
  prob = c((probs[1]+1/3*probs[2]),2/3*probs[2], probs[4]+1/3*probs[3], 2/3*probs[3]),
  model = "Beta-Binomial"
)

#altham binomial theta = 0.5
probs = daltbinom(x = 0:3, p = p, theta = 0.5, n = 3)

altham05_dat = tibble(
  outcome = outcomes,
  prob = c((probs[1]+1/3*probs[2]),2/3*probs[2], probs[4]+1/3*probs[3], 2/3*probs[3]),
  model = "Altham Binomial (theta = 0.5)"
)

#altham binomial theta = 1.5
probs = daltbinom(x = 0:3, p = p, theta = 1.5, n = 3)

altham15_dat = tibble(
  outcome = outcomes,
  prob = c((probs[1]+1/3*probs[2]),2/3*probs[2], probs[4]+1/3*probs[3], 2/3*probs[3]),
  model = "Altham Binomial (theta = 1.5)"
)

#altham beta-binomial theta = 0.5
probs = daltbbinom(x = 0:3, alpha = alpha, beta = beta, theta = 0.5, n = 3)

althambb05_dat = tibble(
  outcome = outcomes,
  prob = c((probs[1]+1/3*probs[2]),2/3*probs[2], probs[4]+1/3*probs[3], 2/3*probs[3]),
  model = "Altham Beta-Binomial (theta = 0.5)"
)

#altham beta-binomial theta = 1.5
probs = daltbbinom(x = 0:3, alpha = alpha, beta = beta, theta = 1.5, n = 3)

althambb15_dat = tibble(
  outcome = outcomes,
  prob = c((probs[1]+1/3*probs[2]),2/3*probs[2], probs[4]+1/3*probs[3], 2/3*probs[3]),
  model = "Altham Beta-Binomial (theta = 1.5)"
)

#put together
ggdat = bind_rows( #had to look this up, i forgar
  binom_dat,
  nbinom_dat,
  bbinom_dat,
  altham05_dat,
  altham15_dat,
  althambb05_dat,
  althambb15_dat
)
```

```r

ggplot(ggdat, aes(x = outcome, y = prob, fill = outcome)) +
  geom_col() +
  facet_wrap(~model) +
  ylab("Probability") +
  xlab("Outcomes") +
  theme_bw()
```

Figure 9: Probabilities for outcomes across models



The binomial model assumes a fixed leg-win probability of 0.7. So we see a higher trend to 2 and 3 wins. In fact, in all the models it is most probable that Player A's record is 2-0 in the game. The negative binomial shows the same idea. The beta-binomial is similar but allows the leg-win probability to vary very slightly to adjust for more realistic variations. The Altham Binomial models show how dependencies between legs affect overall outcome. With lower thetas, wins stack and clump together, creating more 2-0 or 0-2 chances, while the opposite is true for higher thetas, creating more 2-1 or 1-2 chances. Combining the Altham with the Beta Binomial we get the combination of leg dependency and variations in leg-win probability. Overall, each model does better with different scenarios. The Altham Beta-Binomial model does well to show hot-hand behavior with varying performance through a match. But, depending on environment and set up, each model will better illustrate those varying systems.

References

Wolodzko, Tymoteusz. 2026. *extraDistr: Additional Univariate and Multivariate Distributions*. https://doi.org/10.32614/CRAN.package.extraDistr.